

Java Streams API

Stream creation; intermediate & terminal operations; Collectors
API; parallel streams awareness

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Introduction to Java Streams

Stream in Java was introduced in Java 8 to simplify the processing of collections and data. It provides a modern and efficient way to perform operations on groups of objects using a functional programming approach. Streams help developers write cleaner, shorter, and more readable code for data processing tasks.

- Allows processing of data without modifying the original collection.
- Supports parallel processing to improve performance on large datasets.
- Reduces the need for lengthy loops and temporary variables in code.

How to Create a Java Stream

Java Stream Creation is one of the most basic steps before considering the functionalities of the Java Stream. Below is the syntax given for declaring a Java Stream.

- Syntax : `Stream<T> stream;`
`Stream<T>`: general object stream for reference types

Java Stream Features

The features of Java streams are mentioned below:

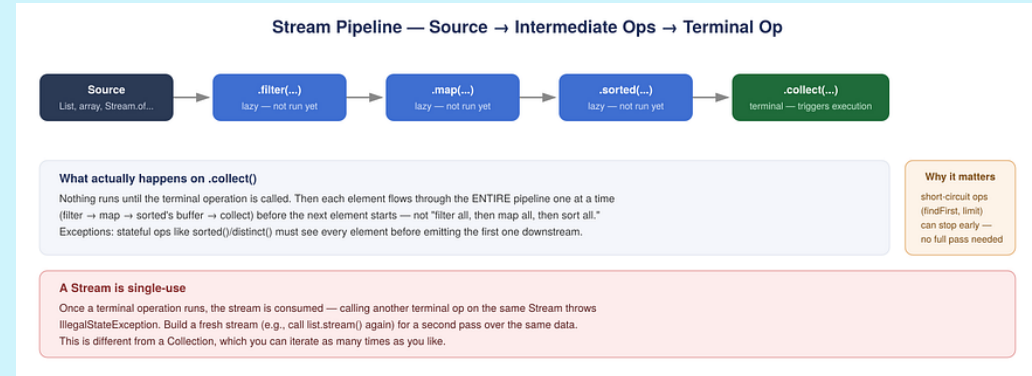
- A Stream is not a data structure; it just takes input from Collections, Arrays or I/O channels.
- Streams do not modify the original data; they only produce results using their methods.
- Intermediate operations (like filter, map, etc.) are lazy and return another Stream, so you can chain them together.
- A terminal operation (like collect, forEach, count) ends the stream and gives the final result.

What is a Stream?

A Stream represents a sequence of elements supporting sequential and parallel aggregate operations.

Important characteristics:

- Introduced in Java 8
- Used mainly with Collections
- Does not store data
- Does not modify the original collection by default
- Supports functional-style programming
- Operations can be chained
- Supports sequential and parallel processing
- Stream operations are generally divided into:
Intermediate operations
Terminal operations



A stream is not a data structure — it holds no elements of its own. It's a **pipeline description**: a source, zero or more lazy transformations, and one operation that finally triggers execution.

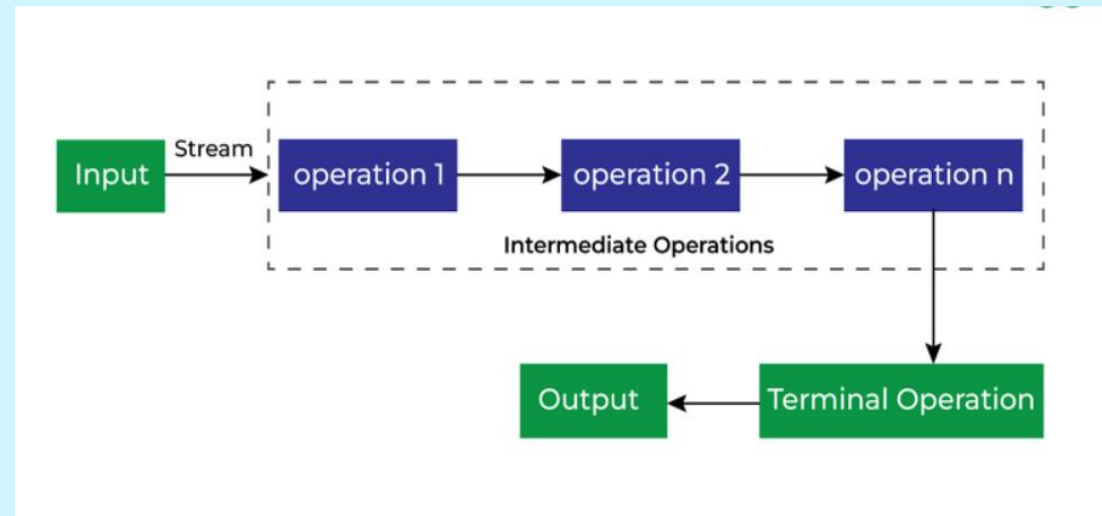
Different Operations On Streams

There are two types of Operations in Streams:

1. Intermediate Operations

- Intermediate operations are the operations that transform a stream into another stream. These operations are lazy in nature, meaning they do not execute until a terminal operation is called.

Intermediate Operations are the types of operations in which multiple methods are chained in a row. Intermediate operations are lazy. `filter()`, `map()`, `sorted()` — none of them do any work. They just build up a recipe.



Characteristics of Intermediate Operations

- Methods are chained together.
- Intermediate operations transform a stream into another stream.
- It enables the concept of filtering where one method filters data and passes it to another method after processing.

Important Intermediate Operations

There are a few Intermediate Operations mentioned below:

1. map(): The map method is used to return a stream consisting of the results of applying the given function to the elements of this stream.

Syntax: `<R> Stream<R> map(Function<? super T, ? extends R> mapper)`

2. filter(): The filter method is used to select elements as per the Predicate passed as an argument.

Syntax:

`Stream<T> filter(Predicate<? super T> predicate)`

3. sorted(): The sorted method is used to sort the stream.

Syntax:

- `Stream<T> sorted()`
- `Stream<T> sorted(Comparator<? super T> comparator)`

Characteristics of Intermediate Operations

4. flatMap(): The flatMap operation in Java Streams is used to flatten a stream of collections into a single stream of elements.

Syntax:

- `<R> Stream<R> flatMap(Function<? super T, ? extends Stream<? extends R>> mapper)`

5. distinct(): Removes duplicate elements. It returns a stream consisting of the distinct elements (according to `Object.equals(Object)`).

- Syntax: `Stream<T> distinct()`

6. peek(): Performs an action on each element without modifying the stream. It returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

- Syntax: `Stream<T> peek(Consumer<? super T> action)`

Summary of Intermediate Operations

Operation	Purpose	Example
<code>filter()</code>	Select elements based on condition	<code>filter(x -> x > 10)</code>
<code>map()</code>	Transform each element	<code>map(x -> x * 2)</code>
<code>flatMap()</code>	Flatten nested streams	<code>flatMap(List::stream)</code>
<code>distinct()</code>	Remove duplicates	<code>distinct()</code>
<code>sorted()</code>	Sort elements	<code>sorted()</code>
<code>peek()</code>	Inspect elements	<code>peek(System.out::println)</code>
<code>limit()</code>	Take first N elements	<code>limit(5)</code>
<code>skip()</code>	Skip first N elements	<code>skip(2)</code>

Summary of Intermediate Operations

Java Stream API – Operations Diagram

Intermediate Operations vs Terminal Operations



Intermediate Operations

These operations return another Stream, so you can chain them together. They are lazy and do not execute until a terminal operation is invoked.

Operation	Purpose	Example
filter()	Select elements based on a condition.	<code>stream.filter(x -> x > 10)</code>
map()	Transform each element using a function.	<code>stream.map(x -> x * 2)</code>
flatMap()	Flatten nested streams into a single stream.	<code>stream.flatMap(List::stream)</code>
distinct()	Remove duplicate elements.	<code>stream.distinct()</code>
sorted()	Sort elements (natural order or custom comparator).	<code>stream.sorted()</code> // or <code>.sorted(Comparator)</code>
peek()	Inspect elements (mainly for debugging or logging).	<code>stream.peek(System.out::println)</code>
limit()	Take the first <i>n</i> elements.	<code>stream.limit(5)</code>
skip()	Skip the first <i>n</i> elements.	<code>stream.skip(2)</code>

Key Points

- Intermediate operations: `filter()`, `map()`, `flatMap()`, `distinct()`, `sorted()`, `peek()`, `limit()`, `skip()` → return a Stream (lazy).
- Terminal operations: `collect()`, `forEach()`, `count()`, `reduce()`, `min()`, `max()`, `anyMatch()`, `allMatch()`, `noneMatch()`, `findFirst()`, `findAny()` → produce a result and end the stream.
- Once a terminal operation is called, the stream is consumed and cannot be reused.

Terminal Operations

These operations produce a final result and terminate the stream pipeline. Once a terminal operation is called, the stream is consumed and cannot be reused.

Operation	Purpose	Example
collect()	Collect the result into a collection (e.g., List, Set, Map).	<code>stream.collect(Collectors.toList())</code>
forEach()	Perform an action for each element.	<code>stream.forEach(System.out::println)</code>
count()	Return the number of elements.	<code>stream.count()</code>
reduce()	Combine elements into a single result.	<code>stream.reduce(0, Integer::sum)</code>
min()	Find the minimum element (using a Comparator).	<code>stream.min(Integer::compareTo)</code>
max()	Find the maximum element (using a Comparator).	<code>stream.max(Integer::compareTo)</code>
anyMatch()	Check if any element matches the condition.	<code>stream.anyMatch(x -> x > 10)</code>
allMatch()	Check if all elements match the condition.	<code>stream.allMatch(x -> x > 10)</code>
noneMatch()	Check if no element matches the condition.	<code>stream.noneMatch(x -> x > 10)</code>
findFirst()	Return the first element (if present).	<code>stream.findFirst()</code>
findAny()	Return any element (useful in parallel streams).	<code>stream.findAny()</code>

Complete Stream Pipeline Example

```
List<String> result = listOfLists.stream()
    .flatMap(List::stream)           // intermediate
    .filter(s -> s.startsWith("S")) // intermediate
    .map(String::toUpperCase)       // intermediate
    .distinct()                     // intermediate
    .sorted()                       // intermediate
    .peek(intermediateResults::add) // intermediate
    .collect(Collectors.toList());  // terminal
```

Intermediate operations build the pipeline → Terminal operation executes it.

Java Streams – Intermediate and Terminal Operations

Intermediate Operations

(Return another Stream, so you can chain them)

Operation	Purpose	Example
filter()	Select elements based on a condition	<code>stream.filter(x -> x > 10)</code>
map()	Transform each element	<code>stream.map(x -> x * 2)</code>
flatMap()	Flatten nested streams	<code>stream.flatMap(List::stream)</code>
distinct()	Remove duplicates	<code>stream.distinct()</code>
sorted()	Sort elements	<code>stream.sorted()</code>
peek()	Perform an action on each element (mainly for debugging)	<code>stream.peek(System.out::println)</code>
limit()	Take first <i>N</i> elements	<code>stream.limit(5)</code>
skip()	Skip first <i>N</i> elements	<code>stream.skip(2)</code>
takeWhile()	Take elements while condition is true (Java 9+)	<code>stream.takeWhile(x -> x < 50)</code>
dropWhile()	Drop elements while condition is true (Java 9+)	<code>stream.dropWhile(x -> x < 50)</code>

Example – Complete Stream Pipeline

```
List<List<String>> list = List.of(
    List.of("Java", "Python"),
    List.of("Spring", "Java", "SQL"));

List<String> result = list.stream()
    .flatMap(List::stream) // intermediate
    .filter(s -> s.startsWith("S")) // intermediate
    .map(String::toUpperCase) // intermediate
    .distinct() // intermediate
    .sorted() // intermediate
    .collect(Collectors.toList()); // terminal

System.out.println(result);
```

Source
(Collection, Array, etc.)
stream()

Intermediate Operations
(0 or more)

- filter()**
- map()**
- flatMap()**
- distinct()**
- sorted()**
- peek()**
- limit()**
- skip()**
- ...

Terminal Operation
(Exactly one)

Result
(List, Set, Optional, Count, Boolean, etc.)

Terminal Operations

(Produce a result or side effect, end the stream)

Operation	Purpose	Example
forEach()	Perform an action on each element	<code>stream.forEach(System.out::println)</code>
collect()	Collect elements into a collection	<code>stream.collect(Collectors.toList())</code>
toList() (Java 16+)	Collect elements into an unmodifiable list	<code>stream.toList()</code>
toArray()	Convert to an array	<code>stream.toArray()</code>
reduce()	Combine elements to a single result	<code>stream.reduce(0, (Integer::sum))</code>
count()	Count elements	<code>stream.count()</code>
anyMatch()	Check if any element matches a condition	<code>stream.anyMatch(x -> x > 10)</code>
allMatch()	Check if all elements match a condition	<code>stream.allMatch(x -> x > 10)</code>
noneMatch()	Check if no element matches a condition	<code>stream.noneMatch(x -> x > 10)</code>
findFirst()	Return the first element (Optional)	<code>stream.findFirst()</code>
findAny()	Return any element (Optional)	<code>stream.findAny()</code>
max()	Return the maximum element (Optional)	<code>stream.max(Comparator.naturalOrder())</code>
min()	Return the minimum element (Optional)	<code>stream.min(Comparator.naturalOrder())</code>

Key Points

- Intermediate operations return another Stream.
- Terminal operations produce a result or side effect.
- You can have zero or more intermediate operations, but exactly one terminal operation.
- Streams are lazy – intermediate operations execute only when a terminal operation is called.
- Once a terminal operation is applied, the stream is closed and cannot be reused.

Comparisons

Criteria	Lambda expression	Method reference	
Purpose	Defines behavior inline	Reuses existing method/constructor	
Syntax	<code>x -> x.trim()</code>	<code>String::trim</code>	
When preferred	Custom logic or multiple operations	Direct forwarding to named method	
Target	Functional interface	Functional interface	
Interface	Returns value?	Primary question	Example
Predicate	boolean	Does it satisfy condition?	<code>n -> n > 18</code>
Function	Yes (R)	What is transformed value?	<code>s -> s.length()</code>
Consumer	No (void)	What action to perform?	<code>System.out::println</code>
Supplier	Yes (T)	What value to provide?	<code>() -> 100</code>

2. Terminal Operations

Terminal operations are the operations that produce the final result from a stream and end the stream processing.

These operations trigger the execution of all intermediate operations.

These Operations are not processed further just return a final result value.

Important Terminal Operations

1. collect(): The collect method is used to return the result of the intermediate operations performed on the stream.

Syntax:

```
<R, A> R collect(Collector<? super T, A, R> collector)
```

2. forEach(): The forEach method is used to iterate through every element of the stream.

Syntax:

```
void forEach(Consumer<? super T> action)
```

3. reduce(): The reduce method is used to reduce the elements of a stream to a single value. The reduce method takes a BinaryOperator as a parameter.

Syntax:

```
T reduce(T identity, BinaryOperator<T> accumulator)
```

```
Optional<T> reduce(BinaryOperator<T> accumulator)
```

2. Terminal Operations

4. count(): Returns the count of elements in the stream.

Syntax:

```
long count()
```

5. findFirst(): Returns the first element of the stream, if present.

Syntax:

```
Optional<T> findFirst()
```

6. allMatch(): Checks if all elements of the stream match a given predicate.

Syntax:

```
boolean allMatch(Predicate<? super T> predicate)
```

7. anyMatch(): Checks if any element of the stream matches a given predicate.

Syntax:

```
boolean anyMatch(Predicate<? super T> predicate)
```